Databases for non-event data in the Mu3e Experiment

Urs Langenegger

Mu3e Internal Note 0133

10th September 2026

This note describes the `CDB` - the database hosting "conditions" (together with low-level "detector configuration" and "detector calibration") data and the `RDB` (the run database) for the Mu3e experiment. The hardware and system setup of the server `mu3edb0`, hosting the databases, is described.

The design of the `CDB` and the `RDB` software, the storage model, the database backends, and the APIs are described.

All implemented calibrations are documented from a technical/algorithmic point of view (no detector performance analysis is provided).

The global tags, with their associated tags and payloads, are described in detail (new features, which ones to use and why others should not be used).

Example workflows for maintaining the `CDB` (in particular creating and validating global tags with their associated tags and payloads), uploading and downloading calibration data files, and maintaining the `RDB` are provided. Uploading and downloading detector configuration files and detector calibration data files is also described.

Contents

Document history:

Ver.	Date	Author	Comment
1.0	January 31, 2026	Urs	Initial version
2.1	August 5, 2026	Urs	detconfigs and detcal sections added

1 Introduction

The data measured by the Mu3e experiment comprises event-data (the data resulting from interactions of charged particles with the sensitive elements of the Mu3e detector), environmental data (for instance, temperature), and operational data (*e.g.*, beam current and magnetic field strength). As first introduced in Ref. [1], a `CDB` has been set up for the Mu3e Experiment to store and provide access to "conditions" data.

It is imperative to realize that in addition to the "conditions", the `CDB` has been expanded to also provide access to

- `RDB`, the "run-database" (e.g. run information),
- "detector configuration" data (e.g. versioned pixel mask files),
- "detector calibration" data (e.g. the results of low-level detector calibration procedures),

The event-data recorded in DAQ runs require for their optimal reconstruction and analysis the corresponding conditions (e.g. which pixel sensor is turned off, changing geometries/alignment, etc.), and possibly also detector configuration or calibration files, with an interval of validity (IOV) that is a priori not coupled to single runs and, ideally, should cover longer periods of multiple runs (stable conditions should be the goal). Figure 1 illustrates the situation for the conditions data. In Section 3, the software architecture and API are described in more detail.

The conditions data (calibration constants) are encoded in "payloads" consisting of descriptive meta-data and and the actual data in the form of (eventually[1]) a binary large object (BLOB).

By definition, a "tag" is the combination of a name (a specific conditions/calibration name plus another identifying/versioning string) and a list of IOVs. An example for a "tag" describing the alignment of the pixel detector could be for instance

```
tag:  pixelefficiency_datav6.3=2025V0 = [1, 100, 200, 5700]
```

where the conditions/calibration name is `pixelefficiency`, and the identifying string indicates which `mu3e` version is required for the usage of this tag together with additional information (version of the alignment payload and that this is for data taken in 2025). The array indicates the runnumbers where the payload changes. The identifying string currently is closely related to the name of the "global tag" (see below) in which the tag was first used. Often, the term "tag" is used for either the tagname or the compound object (tag name together with the array of runnumbers).

By choice, an IOV is open-ended, *i.e.*, the correct IOV for a given run X is the IOV labeled with the largest run number smaller or equal to X. For example, given a tag with IOV list $[1, 236, 567]$, the correct IOV for run 235 would be IOV 1.

Processing the event-data of a specific run requires the conditions/calibration constants for (possibly many) of the involved detector components, possibly for

[1] At the moment, the calibration data is not stored in binary form, but rather base64 encoded. This is an implementation detail that could be changed without affecting the CDB design.

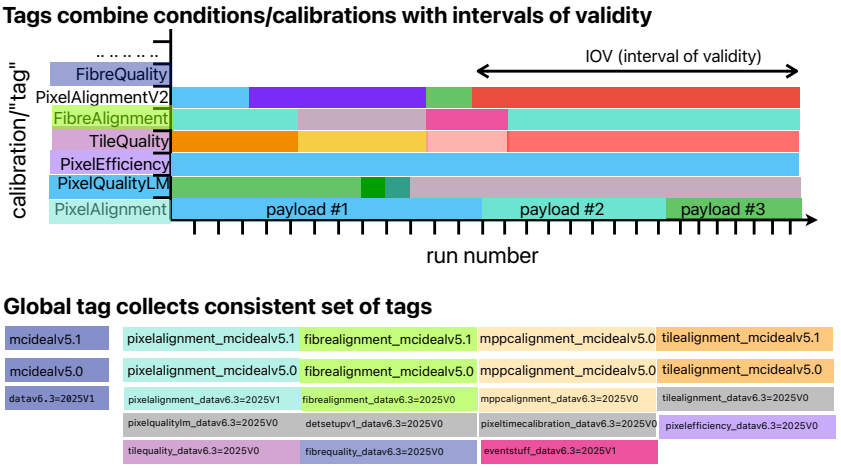

Figure 1: Changing conditions for calibration payloads that need to be managed through the CDB. The time-dependence on the abscissa is indicated through the run numbers, while the labels on the ordinate indicate different calibrations (either different, improved, versions of a calibration, a different type of calibration for the same detector components, or calibrations for different detector components). A payload consists of the calibration constants and descriptive meta-data. Each payload has a specific interval of validity, which is (by design choice) open-ended.

multiple software versions (in case the data of different data taking periods require different software versions). To manage this complex situation, "global tags" (GTs) provide the means to organize tags for specific software versions and tasks: A GT contains a list of tags. In Section 7, the global tags used for the Mu3e Experiment are described in more detail.

A "hash"[2] uniquely labels a payload (given a tag with IOV) in the storage back-end, for example for a "pixel quality" calibration payload of the `mcideal` global tag

```
hash:  tag_pixelqualitylm_mcideal_iov_1
```

Here, `tag` and `iov` are fixed components of the hash string, while `pixelquality`, `mcideal`, and `1` indicate variable components (calibration name with identification string and IOV) and are different for different calibrations. Fig. 2 shows the CDB browser displaying GTs, tags, and payloads.

The design of the database *storage* is kept very simple and is illustrated in Fig. 3. There are three required collections (or tables, in SQL[3] terminology) storing everything related to the conditions data. The three required collections are the "global tags", the "tags" (or "iovs"), and the "payloads". In addition, there are other collections for auxiliary data, *e.g.*, the RDB(collecting

[2]At the moment, the hash is a simple string.

[3]Structured Query Language, often used in the context of relational schema-based databases.

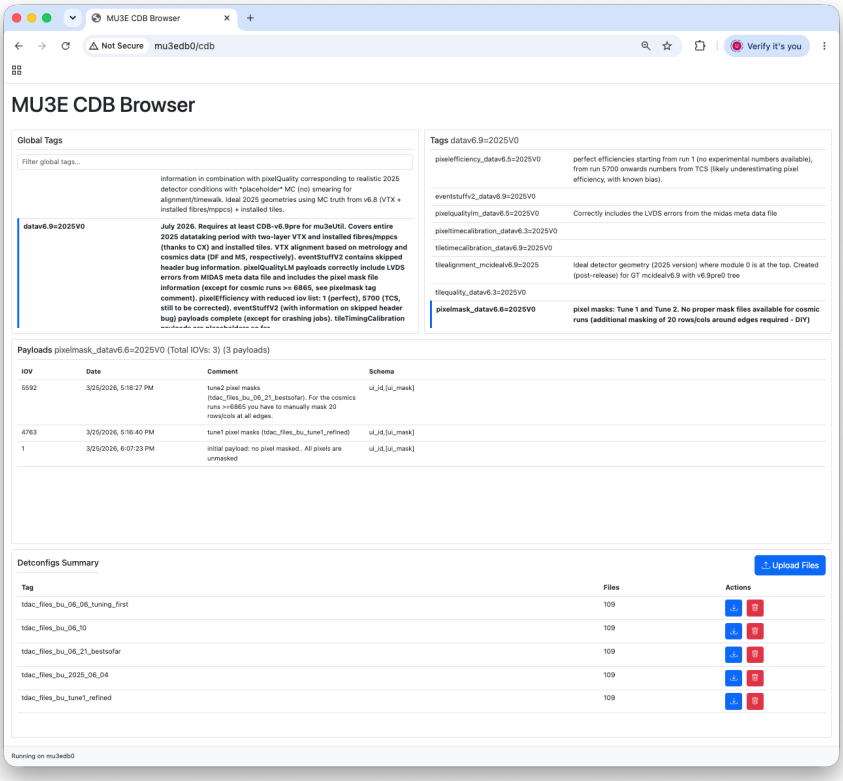

Figure 2: `CDB` browser illustrating a selection of GTs, the tags making up the GT `datav6.9=2025V0` and the payloads referenced in the tag `pixelmask_datav6.6=2025V0`. Clicking on the payload row opens a popup window showing the decoded payload content.

basic information on run like start and end times, number of events, readout configuration, *etc.*) or meta-data (for easier database access).

The database server `mu3edb0` at PSI hosts the storage and database backends for both `CDB` and `RDB` and provides HTTP REST endpoints to store and retrieve detector configuration files (Mongo detconfigs, keyed by tag) and detector calibration payloads (Mongo detcal + GridFS, keyed by name and run, with IOV-style lookup). In addition, it serves as a possible backup server for the partsDB. The hardware is described in Section 2.

Some of the algorithms used for the derivation of the conditions data are described in Section 4. Exemplary usage of the `CDB`, with detailed command line instructions, is described in Section 5. The changes of the `CDB` code with Mu3e releases are documented in Section 6.

The `RDB`, the detector configuration files, and the detector calibration data are described in more detail in Sections 9, 10, and 11, respectively.

The software stack for the `CDB` and anything connected to it is hosted in two repositories, partially with (intended) duplication:

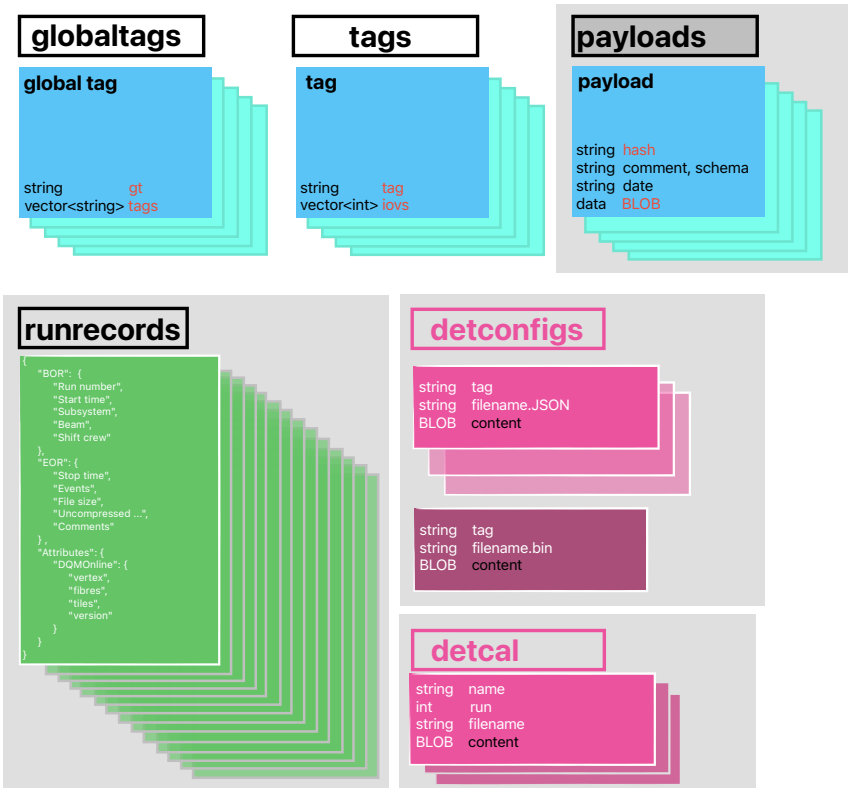

Figure 3: Data model of the CDB. The conditions database comprises the top row, the bottom row indicates collections also hosted by the CDB. The "string" BLOB in the conditions payload is a base64 encoded string, but eventually could be replaced with true binary data. In the detconfigs and detcal collections, the BLOB is of no specified type, possibly ASCII JSON or NDJSON or a binary file.

- The main reference repository is `https://github.com/ursl/mu3eanca`, where the CDB code is in `db0/cdb2` and the database node server is in `db1/rest`. This code base has a MONGODB dependency which must be fulfilled for compilation.
- `https://bitbucket.org/mu3e/mu3eutil/src/dev/cdb/` contains the mu3e CDB code without any MONGODB dependency.

2 Hardware and System Setup

The databases for non-event data are hosted by one server, `mu3edb0`, located in the racks of the Mu3e DAQ room. The machine has 8 cores (Xeon(R) Silver 4309Y CPU at 2.80GHz) and 128GB of RAM. It has two 10G ethernet interfaces, one with a fixed IP address assigned, `129.129.143.6` with MAC address `ec:e7:a7:11:23:58`, in the normal PSI network (*not* the mu3e network!). The second ethernet interface could provide for possible integration into the mu3e network, if required in the future. There is a user account `mu3e` intended for all system work (via `sudo`). The password for this user is written on the server.

`nginx` is configured to redirect

https://mu3edb0.psi.ch/cbd → https://mu3edb0.psi.ch:5050/cbd
https://mu3edb0.psi.ch/rbd → https://mu3edb0.psi.ch:5050/rbd
http://mu3edb0.psi.ch/cbd → http://mu3edb0.psi.ch:5050/cbd
http://mu3edb0.psi.ch/rbd → http://mu3edb0.psi.ch:5050/rbd

https://mu3edb0.psi.ch → https://mu3edb0.psi.ch:3001

The last line is relevant for the partsDB. The relevant configuration files are stored in Ref. [2]

Software RAID (level 1) integrates (1) two 240GB SATAIII SSD disks into the root file system and (2) two 24 TB SASIII HDD disks into two data partitions for the CDB/RDB and the partsDB:

- `/dev/mapper/vg0-lv-0 228422036 16692932 200053048 8% /`
- `/dev/mapper/vg_data-lv_couchdb 2112647088 75730440 1929526084 4% /mnt/data2`
- `/dev/mapper/vg_data-lv_mongodb 21288065024 176686488 21111378536 1% /mnt/data1`

In total 12 SSD/HDD Drive Bays are available, the majority still free for future disk additions.

2.1 CDB and RDB

The user account for all CDB/RDB related work and code is `mu3ecdb`, with the same password as the user `mu3e`. All relevant code for the CDB/RDB node server is located in `/home/mu3ecdb/mu3eanca/db1/rest/.`

A functional CDB/RDB instance requires the following software:

- MongoDB listening on port 27017
- node server listening on port 5050, cf. `mu3eanca/db1/rest/` [3].

2.2 PartsDB

The user account for all CDB/RDB related work and code is `mu3epartsdbadmin`. The password for this user is written on the server. All relevant code for the CDB/RDB node server is located in `/home/mu3epartsdbadmin/mu3epartdb/.`
Important: The PartsDB server running on mu3edb0 is *not* where you should access the PartsDB. Use the original in Heidelberg. Its intention is to provide for a backup server in case anything happens to the server in Heidelberg (though some finetuning to the reverse proxy is required for proper write and database acccess).

3 Software architecture and API

This section describes how non-event data are served to Mu3e software and users. The logical data model (global tags, tags, IOV lists, payload hashes) is summarized in section 1 and in Ref. [1]; here the focus is on components and machine-facing APIs. The interfaces exist outside reconstruction code.

3.1 Architecture

Storage and processes. The primary store is a MongoDB database on the server `mu3edb0` (collections for conditions data, run records, detector configuration file bundles, calibration files, and optional GridFS buckets for large binary objects). An Express application (`mu3eanca/db1/rest`) connects to the same database and exposes JSON over HTTP. Mu3e C++ code does not talk to MongoDB directly for routine access; it uses a small abstraction layer that can target either the REST service or a directory tree of JSON documents (offline use, tests, or air-gapped copies).

Abstract database access (`cdbAbs`). The class `cdbAbs` defines the interface used by conditions and run-configuration logic. Implementations include:

- `cdbJSON` — reads the same document layout from a local directory (one file per document where appropriate);
- `cdbMongo` — native driver access to a MongoDB instance;
- `cdbRest` — HTTP client (`libcurl`) against the Express routes under the `/cdb` prefix (subsection 3.2).

The interface covers: global tags, per–global-tag tag lists, IOV arrays per tag, payload retrieval by hash, configuration documents by hash, and run records by run number, plus helpers to list run numbers. Comments on global tags and tags are exposed for tooling and the web browser.

Conditions coordinator (`Mu3eConditions`). `Mu3eConditions` is the singleton entry point used in reconstruction and analysis. For a chosen global tag it caches tag names and IOV maps, resolves the correct hash for a given run, and notifies registered calibration objects when the run changes. It also exposes run records and registered configuration payloads (`cfgPayload`) alongside calibration access.

Calibration layer (`calAbs` and derivatives). Each calibration type subclasses `calAbs`. The base class handles caching of `payload` objects, hash-based refresh, and file I/O; derived classes implement `calculate`, BLOB encoding and decoding, and typed accessors. The payload "schema" is therefore defined in code, not in the database engine—as in Ref. [1]. Concrete classes are instantiated via `Mu3eConditions::createClass` and `Mu3eCalFactory`.

Writing and maintenance tools. Payload JSON files and bootstrap inputs are produced by standalone programs (e.g. `cdbPayloadWriter`, `cdbRunPayloadWriter`) and sync utilities (`syncJSON`, `syncMongo`, `syncRDB`, `syncCloud`). These are part of the same source tree as the calibration classes (`db0/cdb2`) and are used to populate or mirror the database; they are not required inside event processing.

Figures. The storage-oriented overview (conditions collections, run records, detector-configuration bundles) is Fig. 3 in section 1. The C++ structure is shown in Fig. 4.

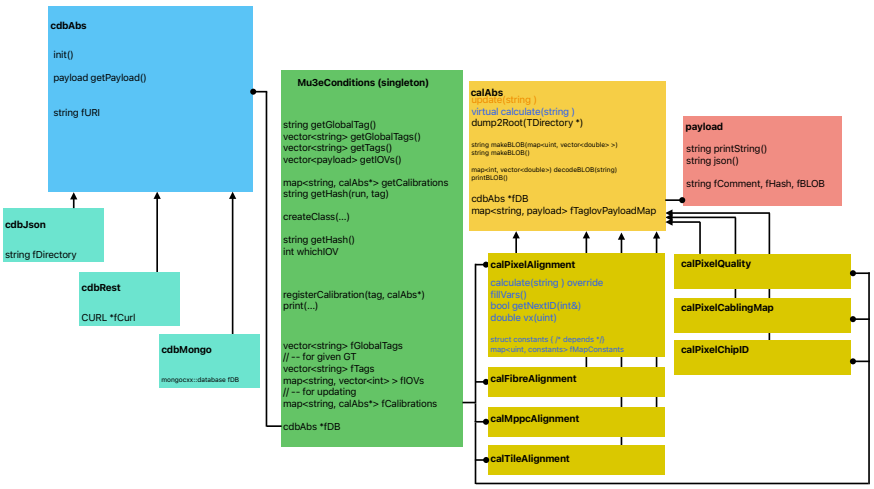

Figure 4: Class organization for Mu3e conditions management and access to database storage and calibration constants (see also Ref. [1]).

3.2 HTTP API for the conditions database

The REST application listens on a configurable port (default `5050`) and mounts route modules as follows: `/cdb` conditions and related helpers, `/detconfigs` detector-configuration file bundles, `/detcal` detector-calibration file storage, `/rdb` run-database pages and run-record ingestion. The C++ client `cdbRest` is configured with the *base URL of the CDB router*, i.e. `http(s)://<host>:<port>/cdb`, not the server root.

Core read pattern for the conditions database. The routes consumed by `cdbRest` follow a uniform pattern:

- `GET /cdb/findOne/<collection>/:id` — return one document or `Not found`. Collections used here include `globaltags` (key: `gt`), `tags` (key: `tag`), `payloads` (key: `hash`), `configs` (key: `cfgHash`), and `runrecords` (key: run number in the URL path, matched against the BOR run-number field).
- `GET /cdb/findAll/<collection>` — list or scan a collection; used for global tags and for the flat list of run numbers (`/cdb/findAll/runNumbers`).

For payloads, large BLOBs may be stored in GridFS; the server merges the grid file into the document as a base64 `BLOB` field before sending the response, so clients always see the same JSON shape.

Authentication. The Express service under `db1/rest` does not enforce HTTP authentication: routes under `/cdb`, `/rdb`, etc. are open to any client that can reach the host and port.

Additional CDB routes (browser and tools). Beyond the minimal `findOne`/`findAll` surface, the CDB router exposes helpers used by the web UI and command-line utilities, including:

- `GET /cdb/findTagsByGlobaltag/:globaltag` — tag metadata and IOV arrays for one global tag;
- `GET /cdb/findPayloadsByTag/:tag` — payload summaries (with optional limit) for hashes belonging to a tag.

The `/detconfigs`, `/detcal`, and `/rdb` APIs (storage model, routes, and further detail) are described in sections 10, 11, and 9, respectively.

3.3 Startup of programs serving the CDB

A working instance needs three long-running pieces, of which only the Express application is started with `pm2`:

- MongoDB on port 27017 (database `mu3e`);
- the Node/Express application `mu3eanca/db1/rest/index.mjs` on port 5050 (section 3.2);
- `nginx`, which reverse-proxies `/cdb`, `/rdb`, `/detconfigs`, `/detcal`, and `/relval` to that port (section 2).

One Node process serves all of those URL prefixes. The RDB, detector configuration, detector calibration, and RelVal dashboards are not separate servers.

On `mu3edb0` the checkout is `/home/mu3ecdb/mu3eanca/db1/rest/` (account `mu3ecdb`). `node`, `npm`, and `pm2` are installed with `nvm`. From that directory:

```
pm2 start index.mjs
pm2 status
pm2 restart index
```

The listen port is `PORT` in the environment (default 5050, loaded from `.env` via `dotenv`). After a reboot, `pm2` must resurrect the saved process list (`pm2 startup` once per machine, then `pm2 save` after a working start).

A second, file-backed browser lives in `db1/restJSON` (default port 5051). It is not required for the MongoDB CDB on `mu3edb0`.

4 Calibration Software

This section describes the software used for calibration of the Mu3e detector. The calibration classes are the classes that are used to provide non-event data (calibration data).

4.1 calAbs

The `calAbs` class is the base class for all calibration classes and provides all functionality in connection to the database and `Mu3eConditions`. Everything requiring a detailed understanding of the payload (schema), is implemented in derived classes.

4.2 calEventStuffV2

The `calEventStuffV2` class is used to provide the CDB access to quantities that could just as well be stored in/accessed from the RDB. The primary use case is for indicating frame numbers where DAQ problems started. For the entire "event" and for each subsystem (pixel, fibres, tiles) it provides the following information:

- `startFrame(Good)Data`
- `endFrame(Good)Data`
- `firstFrameWithFEBProblems`

where the "(Good)" label applies only to the subsystem data. The payload schema is

```
"eventdata.(ull_startframedata,
            ull_endframedata,
            ull_firstframewithfebproblems),
 pixeldata.(ull_startframegooddata,
            ull_endframegooddata,
            ull_firstframewithfebunsortedhitdata),
  tiledata.(ull_startframegooddata,
            ull_endframegooddata,
            ull_firstframewithfebunsortedhitdata),
 fibredata.(ull_startframegooddata,
            ull_endframegooddata,
            ull_firstframewithfebunsortedhitdata)"
```

The user API provides the obvious getter methods for the above quantities.

4.3 calTileTimeCalibration

The `calTileTimeCalibration` class is used to provide the tile time calibration data, consisting of corrections for

- differential non-linearities (DNL): 32 doubles per channel
- time walk (TW): one double per channel

- time alignment (TA): energy dependent doubles stored as vectors with variable (channel dependent) size. The vector with the energy binning is stored in the payload, after the correction constants.

The payload schema is

```
"ui_id,d_dnl[32],d_ta,i_tw_nbins[,d_tw_ns][,d_tw_energy]"
```

The user API provides the following methods:

- `double getTimeAlignmentOffsetNS(uint32_t id)`
- `double getDNLCorrectedTimeFraction(uint32_t id, int ibin)`
- `double getTimeWalkCorrectionNS(uint32_t id, double energy)` This function does a lookup of the correct energy bin (initializing the bin search assuming an equidistant binning).
- `vector<double> getTimeWalkCorrectionNS(uint32_t id)`
- `vector<double> getTimeWalkCorrectionEnergy(uint32_t id)`
- `vector<double> getDNLCorrectedTimeFraction(uint32_t id)`

It should be noted that the `energy` argument in `getTimeWalkCorrectionNS` has the same units as the "energy" in the "timewalk_correction" section of the JSON file.

4.4 calPixelQualityLM

The `calPixelQualityLM` class provides per-chip pixel quality information for the MuPix sensors (tag prefix `pixelqualitylm_`). Unlike a dense quality map for every pixel, the payload stores sparse defect lists: link-level status words, a list of bad columns, and a list of bad pixels. The name suffix "LM" refers to this link-map encoding (three data-link words plus an M-link word).

Payload schema.

```
"ui_id,ui_ckdivend,ui_ckdivend2,ui_linkA,ui_linkB,ui_linkC,ui_linkM,
 i_ncol[,i_col,ui_qual],i_npix[,i_col,i_row,ui_qual]"
```

Per chip: sensor `id`; clock-divider settings `ckdivend`/`ckdivend2`; status words for links A, B, C (column ranges 0–87, 88–171, 172–255) and M (`linkM` encodes an overflow-related rate as `nhit/ovfl`); then `ncol` pairs `(icol, iqual)` and `npix` triples `(icol, irow, iqual)`. Only defective columns/pixels appear in those lists; absent entries are treated as `Good`.

Status codes. Quality values are the `calPixelQualityLM::Status` enum (also documented by `getStatusDocumentation()`):

```
-1=ChipNotFound, 0=Good, 1=Noisy, 2=Suspect, 3=DeclaredBad,
4=LVDSErrorLink, 5=LVDSErrorOtherLink, 6=LVDSErrorTopBottomEdge,
7=DeadChip, 8=NoHits, 9=Masked
```

Pixel lookup (`getStatus`) applies precedence: non-zero link status for the column's link → column status → optional `calPixelMask` mask → per-pixel map entry (default `Good`). Helpers such as `isLinkDead` / `isChipDead` treat `DeadChip`, `NoHits`, and `Masked` as dead; `isLinkBad` is true for any non-zero link status. `getNpixWithStatus` expands link/column defects to equivalent pixel counts (using the link column widths 89/84/83).

User API (selection).

- `Status getStatus(chipid, icol, irow)`
- `Status getColStatus(chipid, icol)`
- `Status getLinkStatus(chipid, ilink)`
- `bool isLinkDead(...)` / `isChipDead(...)` / `isLinkBad(...)`
- `int getNpixWithStatus(...)` / `getNcolWithStatus(...)`
- `int getCkdivend(...)` / `getCkdivend2(...)`
- `double getLVDSOverflowRate(chipid)`
- `readCsv` / `writeCsv` for the human-editable CSV form of the same schema

Payload production (`fillQualityPixels`). Offline payloads are produced by `mu3eanca/run2025/analysis/fillQualityPixels.cc` from merged DQM hitmaps (`hitmap_perChip_*` in a ROOT file), together with MIDAS meta information and a conditions global tag for chip geometry / clock settings. For each chip in a fixed layer-1/2 chip-ID list the program:

1. **Links** (`determineBrokenLinks`), using the chip hitmap together with per-ASIC MIDAS/PCLS meta (`abcLinkMask`, `abcLinkErrs`):
   - Link A/B/C hit integrals are taken over column bins 1–88, 89–172, and 173–256 (rows 1–250). If all three integrals are < 1, the chip is marked `DeadChip` (7) on every link.
   - A chip with all links masked (`abcLinkMask` all 0 or all 9) is stored as `Masked` (9) on A/B/C.
   - Pathological hitmaps where a live link has its mean row near the top or bottom edge ($\langle y \rangle < 10$ or > 245) are flagged `LVDSErrorTopBottomEdge` (6) on links with hits, and `NoHits` (8) or `Masked` (9) on empty/masked links.
   - Per link: zero hits → `NoHits` (8); mask 0 or 9 → `Masked` (9); `abcLinkErrs`$> 10 \rightarrow$ `LVDSErrorLink` (4). If a link has `abcLinkErrs`$>$ 10 while still enabled (`mask`$= 1$), that link stays (4) and the other non-masked links are set to `LVDSErrorOtherLink` (5).
   - The M-link word stores $\lfloor N_{\text{hit}}/N_{\text{ovfl}} \rfloor$ when the hitmap overflow bins (rows 251–252) are non-zero (else 0); a small hand-curated bad-chip list can force `DeclaredBad` (3) on A/B/C.

2. **Columns** (`determineDeadColumns`; skipped when the run is classified as non-beam): for each column not already covered by a non-zero A/B/C link status, the column hit integral (all rows) is compared to a minimum count `minHits`. In the present code `minHits` is 1 (an earlier mean-based threshold is overwritten). Columns with fewer hits than `minHits` are entered in the sparse column list with quality `NoHits` (8). Columns inside a link already flagged bad are omitted (the link status covers them).

3. **Pixels** (`determineNoisyPixels`): flags `Noisy` pixels with hit counts above $\langle N \rangle + 10\,\sigma$ (with $\sigma = \sqrt{\langle N \rangle}$, $\langle N \rangle$ the mean hits over pixels that fired); columns/links already bad are skipped. If more than 40 pixels in a column are `Noisy`, the whole column is marked `Suspect` and those pixels are dropped from the pixel list.

Results are written as CSV (`csv/pixelqualitylm-run<run>.csv`), read back with `calPixelQualityLM::readCsv`, and turned into a CDB payload with hash `tag_pixelqualitylm_<gt>_iov_<run>`. A bootstrap payload for IOV 1 can be created with `fillQualityPixels -p 4` (all chips initially "turned on" / `Good` links). Example:

```
./bin/fillQualityPixels -i datav6.3=2025V0 -g datav6.4=2025V0 \
  -j ~/data/mu3e/cdb/ \
  -f ~/data/mu3e/run2025/mlzr/merged-dqm_histos_06163.root \
  -r datav6.4=2025V0-MidasMeta2025Data.root
```

5 Workflows and tools for the conditions database

5.1 Overview

All commands in this section are in `mu3eUtil/cdb` or in `mu3eanca/db0/cdb2`. Make sure that your `PATH` includes the corresponding directories for their executables. While we do provide examples for each workflow, we do not cover all possible aspects—check the source code for more details.

- CDB global tag listing: `cdbSummaryGT` provides summary information about global tags, tags, and payloads. It allows filtering and searching.
- Payload creation: The primary tool for creating and adding payloads to the CDB is `cdbRunPayloadWriter`, a wrapper for `cdbPayloadWriter`.
- Tag creation/IOV insertion: `cdbInsertIovTag` writes (creates or modifies) the tag JSON file.
- Payload inspection: `cdbPrintPayload` provides textual output of the payload.
- CDB mongoDB upload: `syncMongo`
- CDB mongoDB download: `syncJSON`

5.2 Lists for detector element IDs

The list of detector element IDs is stored in the `cdbPayloadWriter` source code. This is a feature. Currently, the following lists are available:

- MuPix11 sensor IDs for 2025 (two-layer VTX), 2026 (two-layer VTX plus central Layer 3), and the complete detector
- Fibres and MPPC IDs for 2025 (two modules) and the complete detector
- Tile IDs for 2025 and the complete detector

These lists are produced, for instance, with invocations like

```
cdbRunPayloadWriter -m createsensorids
  -f ascii/mu3e_alignment_mcidealv6.5.root
```

Most of the methods can also read from alternative input files, e.g., from a CSV or JSON file. (And yes, this is very old-fashioned, but it is a reproducible method and the input files are kept for reference.)

5.3 New tag with new initial payload(s)

A new tag (most often) implies a new payload, which needs to be created and added to the CDB.

5.3.1 Alignment/geometry payload with root alignment input

It is possible that changes in mu3eSim and/or mu3eTriRec require new payloads, e.g., if the numbering scheme changes, if the orientation of the local coordinate system changes, etc. An incomplete subdetector is another case where a new tag plus payload is required.

- All new alignment/geometry payloads

```
cdbRunPayloadWriter -c alignment
  -f ascii/mu3e_alignment_mcidealv6.8.root
  -a "some good annotation string"
  -p ~/data/mu3e/cdb/payloads
  -t mcidealv6.9
```

- One new alignment/geometry payloads, for a specific era (defined in `cdbPayloadWriter`)

```
cdbRunPayloadWriter -c fibrealignment
  -m 2025
  -f ascii/mu3e_alignment_mcidealv6.5.root
  -a "as installed in 2025"
  -p ~/data/mu3e/cdb/payloads
  -t datav6.5=2025V1

cdbRunPayloadWriter -c tilealignment
  -f ascii/mu3e_alignment_mcidealv6.9pre0.root
  -p ./payloads -t mcidealv6.9
  -a "puts module 0 at the top"

cdbRunPayloadWriter -c tilealignment
  -f ascii/mu3e_alignment_mcidealv6.9pre0.root
  -p ./payloads -t mcidealv6.9=2025
  -a "puts module 0 at the top (2025 version)"
  -m 2025
```

  This will use the list of `IDs` in the `cdbPayloadWriter` configuration to filter the input of the alignment tree such that only the relevant detector elements are included in the payload.

The same approach can be used for an updated alignment/geometry tree for a specific subdetectorin a root file, e.g., from detector alignment with tracks or other means (metrology).

5.3.2 New payloads based on many input files in a directory

A new tag for eventStuffV2 provides an examples how to proceed with many input files prepared in a single directory. The input files need to follow specific naming conventions like `run06219.mid.lz4.json`, `pixelqualitylm-run6755.csv`, etc.

```
./bin/cdbRunPayloadWriter
 -c eventstuffv2
 -d ~/data/mu3e/run2025/rawv2
```

```
-a "includes skipped header bug for pixels, but nothing on fibres/tiles"
-t datav6.9=2025V0
```

This will produce also the payload for run 1 (with perfect settings). If these payloads are driven by a new class, e.g., `calEventStuffV2`, all other GTs for this release need to reflect this change. Therefore create, e.g., a new `mcidealv6.9` GT with

```
./bin/cdbRunPayloadWriter -c eventstuffv2
 -a "ideal tag, for inclusion of skipped header bug for pixels"
 -f ascii/eventstuffv2-ideal.json
 -t mcidealv6.9
 -r 1
```

5.3.3 New payloads from detector input and initial payload

A new tag (most often) implies new payload information from subsystems and a new initial payload (to be created manually). If the new tag is for a partially installed detector, the payloads for run 1 and the subsequent IOVs should match in size.

- First create the payloads from JSON/CSV input files created by the subsystems

```
./bin/cdbRunPayloadWriter -c tiletimecalibration
  -t datav6.9=2025V0
  -f ~/Downloads/calibration_run03274_config.json
  -a "First (placeholder) version. Dead channels w/o constants."
  -i 3274
```

- Then create the corresponding payload for run 1 (with the same size)

```
./bin/cdbRunPayloadWriter -m tiletimecalibration-ideal-2025
  -f ascii/tiletimecalibration-ideal-2025.json

./bin/cdbRunPayloadWriter -c tiletimecalibration
  -t datav6.9=2025V0
  -f ascii/tiletimecalibration-ideal-2025.json
  -a "Created (initially) for GT datav6.9=2025V0"
  -i 1
  -m 2025

./bin/cdbRunPayloadWriter -m pixelmask-ideal -a "ideal tag, for completeness only"
  -t mcidealv6.9 -r 1
```

- Once you have assembled all payloads for the new tag, move the directory with the payloads to the CDB directory and insert the tag with the `cdbInsertIovTag` tool (see next subsection 5.3.4).

5.3.4 Insert new tag with new (initial) payload(s)

The new tag can be inserted with the `cdbInsertIovTag` tool.

```
cdbInsertIovTag -t mppcalignment_datav6.5=2025V1
   -j /Users/ursl/data/mu3e/cdb
   -d ~/data/mu3e/cdb/payloads/mppcalignment_datav6.5=2025V1
   -p mppcalignment_datav6.5=2025V1
```

5.3.5 Upload new tag with payloads to mongoDB

The new tag with payloads can be uploaded to mongoDB using the `syncMongo` tool.

```
syncMongo --dir ~/data/mu3e/cdb
   --host $DBHOST --sync tag
   --name mppcalignment_datav6.5=2025V1
   --deep --del
```

The `-deep` option ensures that all referenced payloads are uploaded as well, and the `-del` option deletes the tag from the mongoDB database, if it already exists there. `$DBHOST` is the host name of the mongoDB server: initially this should be set to `localhost`, and after all checks are done, it should be set to the actual host name, usually `mu3edb0.psi.ch`.

5.3.6 Upload new global tag to mongoDB

The new global tag can be uploaded to mongoDB using the `syncMongo` tool.

```
syncMongo --dir ~/data/mu3e/cdb
--host $DBHOST --sync gt
--name mcrealisticv6.6=2025V1 --del
```

The `-del` option deletes the GT from the mongoDB database, if it already exists there. `$DBHOST` is the host name of the mongoDB server: initially this should be set to `localhost`, and after all checks are done, it should be set to the actual host name, usually `mu3edb0.psi.ch`.

5.3.7 Download GT and tags with payloads from mongoDB

The GT and tags with payloads can be downloaded from mongoDB using the `syncJSON` tool.

```
setenv CDB /Users/ursl/data/mu3e/test-cdb

syncJSON --cdb --dir $CDB --host mu3edb0
   -m tag -e mppcalignment_datav6.5=2025V1 --deep

syncJSON --cdb --dir $CDB --host mu3edb0
   -m tag -e fibrealignment_datav6.5=2025V1 --deep

syncJSON --cdb --dir $CDB --host mu3edb0
   --sync gt --name mcrealisticv6.5=2025V1
```

6 Changes to the CDB code

6.1 mu3eUtil v6.9

- calEventStuffV1 is deprecated.
- calEventStuffV2 providing information on skipped header bug for pixels, but not for fibres of tiles. It is strongly recommended to query the validity of the payload using the new method `isValid(int runNumber)`. The payload does not make sense if its IOV is different from the run number.
- calTileTimeCalibration providing tile time calibration data (first draft, calibration data to be corrected)

6.2 mu3eUtil v6.7

- added small node serve to serve cdbJSON on local machine to browser
- big rewrite for cdbPayloadWriter (uses cdbPayloadWriter)
- added -n (no payload reading as this can be slow) to cdbSummaryGT, the ultimate online documentation for GTs, tags, and more in the CDB

6.3 mu3eUtil v6.6

- added cdbSummaryGT as CLI interface to CDB contents, with filtering possibilities etc
- calPixelTimeCalibration is no longer constrained to be 108 chips
- calPixelMask: new calibration class for pixel mask access (normally very different IOVs than pixelquality, hence different tag/payload)
- calPixelQualityLM will include calPixelMask (in case it's present)
- add new method getPayloadSize(), returning number of detector elements in payload
- add downloadDetConfig (REST api)

6.4 mu3eUtil v6.5

- can manually replace a tag with a new one using the implementation of replaceTag()
- add mu3e::conf.cdb.replaceTags
- add subdirectories for payload storage (keep backward compatibility)
- api migration (remove unused gt from cdb* instantiation)
- runblocks also for runrecords
- change default GT and CDB location to correspond to merlin7
- add eventData.endFrame

- new approach to DB initialization
- utilities for adding/reading comments for GTs and tags
- remove old-style casts, fix compiler warnings

7 Global Tags

A "global tag" (GT) is a collection of tags. Global tags provide a complete and coherent set of tags for the analysis of detector data and MC samples (both in an idealized and a realistic setting). A tag is a name and a collection of IOVs. An IOV is an interval of validity, with one payload per IOV. A payload is a collection of calibration constants.

GTs for detector data have `data` and the running period (e.g. `2025`, but this can change to accomodate "eras", e.g. `2026A`, `2026B`, etc.) in their name and contain all calibration and geometry constants required for an optimal analysis of the detector data. For unstable detector conditions, the IOV list will be quite long.

GTs for MC samples have `mc` in their name and come in two variants:

- `mcideal` for the ideal MC setup. This by convention means the full detector as present in the standard simulation setup.
- `mcrealistic` for non-ideal MC setups. "Non-ideal" means that the simulation setup is missing parts of the detector, to properly describe the detector as installed. In addition, these GTs should contain smeared geometry/alignment information to better describe the detector performance to match the performance of the real detector.

For instance, the information in `pixelqualitylm` will be very different for the two variants: The tag `pixelqualitylm_mcideal` will contain a perfect pixel quality payload (with one IOV), while the tag `pixelqualitylm_mcrealistic` will contain the holes and noisy pixels as present in the real detector with many IOVs.

Similarly, the alignment/geometry information will be different in the three cases: For data, this is the alignment/geometry information obtained with dedicated alignment procedures for the real detector. For `mcideal`, this is the alignment/geometry information from the ideal MC setup. For MC realistic, this is the alignment/geometry information that makes the MC simulation more realistic (this could well be a data tag for the alignment/geometry information, but not for the pixeltiming calibration constants, because here the perfect MC timing must be smeared to describe the data, while the data have to be corrected for timewalk, etc). In the absence of dedicated alignment procedures for the real detector, the alignment/geometry information for `mcrealistic` are taken from the ideal MC setup (filtering out the parts of the detector that are not present in the real detector).

The following considerations should be taken into account when choosing the global tag:

- The global tag must be compatible with the software version. For example, if the software version is v6.3.x, the global tag should be `datav6.3=2025V1` since GTs with a later release in their name may contain tags that are incompatible with the software version. Furthermore changes to reconstruction code may require new tags (e.g. numbering schemes, detector element orientation, etc.) and using the wrong software version will results in incorrect results.

It is often permissible to use a GT with an earlier release in its name, if the tags are compatible with the software version (if only changes to the reconstruction or analysis code have been included in the code base).

- The global tag should be compatible with the data taking period. It is possible that not all tags are produced for all data taking periods.
- The global tag should be compatible with the detector configuration. You should not use a three-layer GT when analyzing two-layer data.
- You should never use a GT with `test` in its names, except for testing purposes.

Table 1: Data global tags for mu3e tagged releases v6.X

GT	Description
`datav6.3=2025V0` Do not use!	November 2025. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics). Alignment from MC. pixelQualityLM payloads are incorrect!
`datav6.3=2025V1` First GT with improved pixel alignment.	December 2025, for BVR2026. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads are incorrect!
`datav6.5=2025V0` Do not use!	March 2026. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file. pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)
`datav6.5=2025V1` First GT with (1) correct LVDS error rate from MIDAS meta data, (2) pixelEfficiency data and (3) fibres alignment information as installed.	June 2026. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file. pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected)

Continued on next page

Table 1 – continued from previous page

GT	Description
`datav6.6=2025V0` Do not use!	March 2026. Requires at least CDB-v6.6pre for mu3eUtil. Covers entire 2025 datataking period with two-layer VTX and fragmentary fibres and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs >= 6865, see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected)
`datav6.6=2025V1` First v6.6 GT with proper fibres/mppcs alignment information as installed.	June 2026. Requires at least CDB-v6.6pre for mu3eUtil. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs >= 6865, see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected)
`datav6.9=2025V0` First GT with (1) eventStuffV2 including "skipped header bug" information and (2) tileTimeCalibration.	June 2026. Requires at least CDB-v6.9pre for mu3eUtil. Covers entire 2025 datataking period with two-layer VTX and installed fibres/mppcs (thanks to CX) and tiles (beam and cosmics, with proper B field). VTX alignment based on metrology and cosmics data (DF and MS, respectively). pixelQualityLM payloads correctly include LVDS errors from MIDAS meta data file and includes the pixel mask file information (except for cosmic runs >= 6865, see pixelmask tag comment). pixelEfficiency with reduced iov list: 1 (perfect), 5700 (TCS, still to be corrected). tileTimeCalibration payloads are placeholders.

In the following, the GTs are described in more detail.

7.1 datav6.3=2025V0

Do not use this tag, it was superseeded by tag `datav6.3=2025V1` in December 2025.
Problems with this tag:

- The pixel quality LM payloads are incorrect since the MIDAS metadata was not correctly parsed.
- The pixel alignment is based on the ideal MC alignment. Also true for the fibre and tile alignment information.

Description of the tag:

```
  GT: datav6.3=2025V0
tags: 11
comment: Do not use! November 2025. Covers entire 2025 datataking
         period with two-layer VTX and fragmentary fibres and tiles
         (beam and cosmics). Alignment from MC. pixelQualityLM
         payloads are incorrect!

 1) pixelalignment_datav6.3=2025V0
    IOV length: 1, comment: VTX only, based on MC
 2) fibrealignment_datav6.3=2025V0
    IOV length: 1,
 3) tilealignment_datav6.3=2025V0
    IOV length: 1
 4) mppcalignment_datav6.3=2025V0
    IOV length: 1
 5) pixelqualitylm_datav6.3=2025V0
    IOV length: 3550
 6) detsetupv1_datav6.3=2025V0
    IOV length: 3
 7) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
 8) tilequality_datav6.3=2025V0
    IOV length: 13
 9) fibrequality_datav6.3=2025V0
    IOV length: 99
10) pixelefficiency_datav6.3=2025V0
    IOV length: 4
11) eventstuffv1_datav6.3=2025V1
    IOV length: 3552
```

7.2 datav6.3=2025V1

This tag is the best tag for 2025 data analysis with mu3e v6.3.x releases, though the pixelQualityLM payloads are not as good as in later GTs (datav6.5=2025VX, etc.). First GT with improved pixel alignment. DF = David Fritz, MS = Mikio Sakurai.
Description of the tag:

```
GT: datav6.3=2025V1
tags: 11
comment: December 2025, for BVR2026. Covers entire 2025 datataking
         period with two-layer VTX and fragmentary fibres and tiles
         (beam and cosmics, with proper B field). VTX alignment based
         on metrology and cosmics data (DF and MS, respectively).
         pixelQualityLM payloads are incorrect!

 1) pixelalignment_datav6.3=2025V1
    IOV length: 1
    comment: VTX only, based on metrology and cosmics data (DF and MS, respectively).
 2) fibrealignment_datav6.3=2025V0
    IOV length: 1
 3) tilealignment_datav6.3=2025V0
    IOV length: 1
 4) mppcalignment_datav6.3=2025V0
    IOV length: 1
 5) pixelqualitylm_datav6.3=2025V0
    IOV length: 3550
 6) detsetupv1_datav6.3=2025V0
    IOV length: 3
 7) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
 8) tilequality_datav6.3=2025V0
    IOV length: 13
 9) fibrequality_datav6.3=2025V0
    IOV length: 99
10) pixelefficiency_datav6.3=2025V0
    IOV length: 4
11) eventstuffv1_datav6.3=2025V1
    IOV length: 3552
```

7.3 datav6.5=2025V0

Do not use this tag, it was superseeded by tag `datav6.5=2025V1` in June 2026.
TCS = Thomas Senger. Problems with this tag:

- The fibre/mppc alignment is based on v6.3 and does not incorporate the changed orientation, etc..

Description of the tag:

```
GT: datav6.5=2025V0
tags: 11
comment: March 2026. Covers entire 2025 datataking period with
         two-layer VTX and fragmentary fibres and tiles (beam and
         cosmics, with proper B field). VTX alignment based on
         metrology and cosmics data (DF and MS, respectively).
         pixelQualityLM payloads correctly include LVDS errors from
         MIDAS meta data file. pixelEfficiency with reduced iov list:
         1 (perfect), 5700 (TCS, to be corrected)

 1) pixelalignment_datav6.3=2025V1
    IOV length: 1
    comment: VTX only, based on metrology and cosmics data (DF and MS, respectively).
 2) fibrealignment_datav6.3=2025V0
    IOV length: 1
 3) tilealignment_datav6.3=2025V0
    IOV length: 1
 4) mppcalignment_datav6.3=2025V0
    IOV length: 1
 5) pixelqualitylm_datav6.5=2025V0
    IOV length: 3330
    comment: Correctly includes the LVDS errors from the midas meta data file
 6) detsetupv1_datav6.3=2025V0
    IOV length: 3
 7) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
 8) tilequality_datav6.3=2025V0
    IOV length: 13
 9) fibrequality_datav6.3=2025V0
    IOV length: 99
10) pixelefficiency_datav6.5=2025V0
    IOV length: 2
    comment: perfect efficiencies starting from run 1 (no experimental
             numbers available), from run 5700 onwards numbers from TCS
             (likely underestimating pixel efficiency, with known bias).
11) eventstuffv1_datav6.3=2025V1
    IOV length: 3552
```

7.4 datav6.5=2025V1

This tag is the correct tag for 2025 data analysis with mu3e v6.5.x releases. First GT with (1) correct LVDS error rate from MIDAS meta data, (2) pixelEfficiency data and (3) fibres alignment information as installed.
Description of the tag:

```
GT: datav6.5=2025V1
tags: 11
comment: June 2026. Covers entire 2025 datataking period with
         two-layer VTX and installed fibres/mppcs (thanks to CX) and
         tiles (beam and cosmics, with proper B field). VTX alignment
         based on metrology and cosmics data (DF and MS,
         respectively). pixelQualityLM payloads correctly include
         LVDS errors from MIDAS meta data file. pixelEfficiency with
         reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)

 1) pixelalignment_datav6.3=2025V1
    IOV length: 1
    comment: VTX only, based on metrology and cosmics data (DF and MS,
             respectively)
 2) fibrealignment_datav6.5=2025V1
    IOV length: 1
 3) tilealignment_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 4) mppcalignment_datav6.5=2025V1
    IOV length: 1
 5) pixelqualitylm_datav6.5=2025V0
    IOV length: 3330
    comment: Correctly includes the LVDS errors from the midas meta data
             file
 6) detsetupv1_datav6.3=2025V0
    IOV length: 3
    comment: Database entry inserted
 7) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 8) tilequality_datav6.3=2025V0
    IOV length: 13
    comment: Database entry inserted
 9) fibrequality_datav6.3=2025V0
    IOV length: 99
    comment: Database entry inserted
10) pixelefficiency_datav6.5=2025V0
    IOV length: 2
    comment: perfect efficiencies starting from run 1 (no experimental
             numbers available), from run 5700 onwards numbers from TCS
             (likely underestimating pixel efficiency, with known bias).
11) eventstuffv1_datav6.3=2025V1
    IOV length: 3552
```

```
comment: Database entry inserted
```

7.5 datav6.6=2025V0

Do not use this tag, it was superseeded by tag `datav6.6=2025V1` in June 2026.
Problems with this tag:

- The fibre/mppcs alignment is based on v6.3 and does not incorporate the changed orientation, etc..

Description of the tag:

```
GT: datav6.6=2025V0
tags: 12
comment: March 2026. Requires at least CDB-v6.6pre for mu3eUtil.
         Covers entire 2025 datataking period with two-layer VTX and
         fragmentary fibres and tiles (beam and cosmics, with proper
         B field). VTX alignment based on metrology and cosmics data
         (DF and MS, respectively). pixelQualityLM payloads correctly
         include LVDS errors from MIDAS meta data file and includes
         the pixel mask file information (except for cosmic runs >=
         6865, see pixelmask tag comment). pixelEfficiency with
         reduced iov list: 1 (perfect), 5700 (TCS, to be corrected)

 1) pixelalignment_datav6.3=2025V1
    IOV length: 1
    comment: VTX only, based on metrology and cosmics data (DF and MS,
             respectively).
 2) fibrealignment_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 3) tilealignment_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 4) mppcalignment_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 5) pixelqualitylm_datav6.5=2025V0
    IOV length: 3330
    comment: Correctly includes the LVDS errors from the midas meta data
             file
 6) pixelmask_datav6.6=2025V0
    IOV length: 3
    comment: pixel masks: Tune 1 and Tune 2. No proper mask files
             available for cosmic runs (additional masking of 20
             rows/cols around edges required - DIY)
 7) detsetupv1_datav6.3=2025V0
    IOV length: 3
    comment: Database entry inserted
 8) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 9) tilequality_datav6.3=2025V0
```

```
    IOV length: 13
    comment: Database entry inserted
10) fibrequality_datav6.3=2025V0
    IOV length: 99
    comment: Database entry inserted
11) pixelefficiency_datav6.5=2025V0
    IOV length: 2
    comment: perfect efficiencies starting from run 1 (no experimental
             numbers available), from run 5700 onwards numbers from TCS
             (likely underestimating pixel efficiency, with known bias).
12) eventstuffv1_datav6.3=2025V1
    IOV length: 3552
    comment: Database entry inserted
```

7.6 datav6.6=2025V1

This tag is the correct tag for 2025 data analysis with mu3e v6.6.x releases.
First v6.6 GT with proper fibres/mppcs alignment information as installed.
Description of the tag:

```
GT: datav6.6=2025V1
tags: 12
comment: June 2026. Requires at least CDB-v6.6pre for mu3eUtil.
         Covers entire 2025 datataking period with two-layer VTX and
         installed fibres/mppcs (thanks to CX) and tiles (beam and
         cosmics, with proper B field). VTX alignment based on
         metrology and cosmics data (DF and MS, respectively).
         pixelQualityLM payloads correctly include LVDS errors from
         MIDAS meta data file and includes the pixel mask file
         information (except for cosmic runs >= 6865, see pixelmask
         tag comment). pixelEfficiency with reduced iov list: 1
         (perfect), 5700 (TCS, to be corrected)

 1) pixelalignment_datav6.3=2025V1
    IOV length: 1
    comment: VTX only, based on metrology and cosmics data (DF and MS,
             respectively)
 2) fibrealignment_datav6.5=2025V1
    IOV length: 1
 3) tilealignment_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 4) mppcalignment_datav6.5=2025V1
    IOV length: 1
 5) pixelqualitylm_datav6.5=2025V0
    IOV length: 3330
    comment: Correctly includes the LVDS errors from the midas meta data
             file
 6) pixelmask_datav6.6=2025V0
    IOV length: 3
    comment: pixel masks: Tune 1 and Tune 2. No proper mask files
             available for cosmic runs (additional masking of 20
             rows/cols around edges required - DIY)
 7) detsetupv1_datav6.3=2025V0
    IOV length: 3
    comment: Database entry inserted
 8) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
    comment: Database entry inserted
 9) tilequality_datav6.3=2025V0
    IOV length: 13
    comment: Database entry inserted
10) fibrequality_datav6.3=2025V0
    IOV length: 99
    comment: Database entry inserted
```

```
11) pixelefficiency_datav6.5=2025V0
    IOV length: 2
    comment: perfect efficiencies starting from run 1 (no experimental
             numbers available), from run 5700 onwards numbers from TCS
             (likely underestimating pixel efficiency, with known bias).
12) eventstuffv1_datav6.3=2025V1
    IOV length: 3552
    comment: Database entry inserted
```

7.7 datav6.9=2025V0

This tag is the correct tag for 2025 data analysis with mu3e v6.9.x releases. It is the first GT with (1) eventStuffV2 including "skipped header bug" information and (2) tileTimeCalibration. The naming convention for the alignment tags for all subdetectors except the VTX changed consistently to `mcidealv6.9=2025` to indicate that this corresponds to unsmeared geometry/alignment information for the installed detector components.

The tilealignment payloads were updated on July 7, 2026, after the announcement of the freezing of v6.9preV0 to be aligned with the contents of the alignment tree of that release (fixing the issue with the module 0 not being at the top). This affects the GTs `datav6.9=2025V0`, `mcrealisticv6.9=2025V0`, and `mcrealisticv6.9=2025V1`.
Description of the tag:

```
GT: datav6.9=2025V0
tags: 13
comment: July 2026. Requires at least CDB-v6.9pre for mu3eUtil.
         Covers entire 2025 datataking period with two-layer VTX and
         installed fibres/mppcs (thanks to CX) and installed tiles.
         VTX alignment based on metrology and cosmics data (DF and
         MS, respectively). eventStuffV2 contains skipped header bug
         information. pixelQualityLM payloads correctly include LVDS
         errors from MIDAS meta data file and includes the pixel mask
         file information (except for cosmic runs >= 6865, see
         pixelmask tag comment). pixelEfficiency with reduced iov
         list: 1 (perfect), 5700 (TCS, still to be corrected).
         eventStuffV2 (with information on skipped header bug)
         payloads complete (except for crashing jobs).
         tileTimingCalibration payloads are placeholders so far.

1) detsetupv1_datav6.3=2025V0
   IOV length: 3
   comment: Database entry inserted
2) eventstuffv2_datav6.9=2025V0
   IOV length: 6035
3) fibrealignment_mcidealv6.9=2025
   IOV length: 1
   detector units: 754
4) fibrequality_datav6.3=2025V0
   IOV length: 99
   detector units: 96
   comment: Database entry inserted
5) mppcalignment_mcidealv6.9=2025
   IOV length: 1
   detector units: 4
6) pixelalignment_datav6.3=2025V1
   IOV length: 1
   detector units: 108
   comment: VTX only, based on metrology and cosmics data (DF and MS,
            respectively)
```

```
 7) pixelefficiency_datav6.5=2025V0
    IOV length: 2
    detector units: 108
    comment: perfect efficiencies starting from run 1 (no experimental
             numbers available), from run 5700 onwards numbers from TCS
             (likely underestimating pixel efficiency, with known bias).
 8) pixelmask_datav6.6=2025V0
    IOV length: 3
    detector units: 108
    comment: pixel masks: Tune 1 and Tune 2. No proper mask files
             available for cosmic runs (additional masking of 20
             rows/cols around edges required - DIY)
 9) pixelqualitylm_datav6.5=2025V0
    IOV length: 3330
    detector units: 108
    comment: Correctly includes the LVDS errors from the midas meta data
             file
10) pixeltimecalibration_datav6.3=2025V0
    IOV length: 1
    detector units: 108
    comment: Database entry inserted
11) tilealignment_mcidealv6.9=2025
    IOV length: 1
    detector units: 1248
    comment: Ideal detector geometry (2025 version) where module 0 is at
             the top. Created (post-release) for GT mcidealv6.9 with
             v6.9pre0 tree
12) tiletimecalibration_datav6.9=2025V0
    IOV length: 2
    detector units: 1248
    comment: First placeholder version
13) tilequality_datav6.3=2025V0
    IOV length: 13
    detector units: 5824
    comment: Database entry inserted
```

7.8 datav7.1=2026V0

This tag is the correct tag for 2026 data analysis with mu3e v7.1.x releases.
Description of the tag:

```
GT: datav7.1=2026V0
tags: 13
comment: Created (initially) for GT datav7.1=2026V0 with startGT

 1) detsetupv1_datav7.1=2026V0
    IOV length: 1
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 2) eventstuffv2_datav7.1=2026V0
    IOV length: 1
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 3) fibrealignment_datav7.1=2026V0
    IOV length: 1
    detector units: 4524
    size: 4524 detector units
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 4) fibrequality_datav7.1=2026V0
    IOV length: 1
    detector units: 4524
    size: 4524 detector units    agrees with alignment size
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 5) mppcalignment_datav7.1=2026V0
    IOV length: 1
    detector units: 24
    size: 24 detector units
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 6) pixelalignment_datav7.1=2026V0
    IOV length: 1
    detector units: 516
    size: 516 detector units
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 7) pixelefficiency_datav7.1=2026V0
    IOV length: 1
    detector units: 516
    size: 516 detector units    agrees with alignment size
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 8) pixelmask_datav7.1=2026V0
    IOV length: 1
    detector units: 516
    size: 516 detector units    agrees with alignment size
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
 9) pixelqualitylm_datav7.1=2026V0
    IOV length: 1
    detector units: 516
    size: 516 detector units    agrees with alignment size
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
10) pixeltimecalibration_datav7.1=2026V0
```

```
    IOV length: 1
    detector units: 516
    size: 516 detector units    agrees with alignment size
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
11) tilealignment_datav7.1=2026V0
    IOV length: 1
    detector units: 2912
    size: 2912 detector units
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
12) tilequality_datav7.1=2026V0
    IOV length: 1
    detector units: 2912
    size: 2912 detector units    agrees with alignment size
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
13) tiletimecalibration_datav7.1=2026V0
    IOV length: 1
    detector units: 2912
    size: 2912 detector units
    comment: Created (initially) for GT datav7.1=2026V0 with startGT
```

8 Tags

8.1 Alignment tags

Alignment[4] tags differ from other calibration tags in the sense that contain absolute coordinates, while other calibration tags contain relative corrections. Pixeltiming calibration constants for `data` should correct as much as possible for imperfect timing measurements (timewalk, biases, etc.), while for `mcrealistic` they should be smear the perfect MC timing to match the data. Correspondingly, the alignment/geometry information for `mcrealistic` can be taken from the corresponding `data` information, but this is not possible for other calibration constants.

[4]Instead of alignment tags we should use the term "geometry" tags, but since this naming scheme has been historically established, it will not be changed.

9 RDB - Run Database

The RDB stores run-dependent information on the Mu3e hardware and data taking. Documents live in the MongoDB collection `runrecords` on `mu3edb0`. Reconstruction and analysis code typically read a run record through `cdbAbs::getRunRecord` (and list run numbers via `getAllRunNumbers`), which use the `/cdb` routes; the dedicated RDB web and maintenance API is mounted at `/rdb`.

9.1 Structure

Each run is one document, keyed by the BOR run number (`BOR."Run number"`). A document typically contains begin- and end-of-run blocks (`BOR`, `EOR`), optional `Attributes` (including significance and classification), `History` entries, and optional `Resources` (attachments stored via GridFS bucket `uploads`).

9.2 Access

The Express router `db1/rest/routes/rdb.mjs` is mounted at `/rdb`. Typical base URL (reverse-proxied on `mu3edb0`): `http://mu3edb0/rdb`. The C++ conditions layer reads the same collection through `/cdb/findOne/runrecords/:id` and `/cdb/findAll/runNumbers`; the `/rdb` module provides paginated and filtered HTML/JSON views for browsers and operators, and write paths for ingestion and bookkeeping. Notable routes include:

- `PUT /rdb/runrecords` — insert or replace the document for one run (JSON body; existing documents with the same BOR run number are deleted first);
- `GET /rdb/` — paginated/filtered run list (HTML or JSON depending on `Accept`);
- `GET /rdb/run/:id` and `GET /rdb/run/:id/json` — single-run page or JSON document;
- `GET /rdb/allRunNumbers`, `GET /rdb/lastRunNumber` — run-number helpers;
- resource upload/download/delete under `/rdb/addResource/:id`, `/rdb/resource/...`, etc.;
- bulk update helpers under `/rdb/bulk/...` (significance, comments, class).

Details of DQM linkage and operational workflows belong elsewhere; the important split is that "CDB" (`/cdb`) and "RDB" (`/rdb`) URL spaces must not be confused even though both may touch `runrecords`.

9.3 Command-line interface

Offline copies of the run records are a JSON directory tree (the same layout read by `cdbJSON`). The Python script `rdb.py` (`mu3eanca/db0/cdb2/rdb.py`) queries that tree without a local MongoDB instance. It depends only on the Python 3 standard library. The intended use is to obtain a list of run numbers that satisfy a set of filters (significant runs, run class, data-quality flags, event count, ...).

```
usage: rdb.py [-h] [--version] [--update] [--dir DIR] [--host HOST] [--sync]
              [-a] [--rebuild] [--significant] [--not-significant]
              [--class NAME] [--dq EXPR] [--min-events N] [--max-events N]
              [-f N] [-l N] [-r LIST] [--comment TEXT] [--shift TEXT]
              [--components TEXT] [--components-out TEXT] [--format FMT]
              [--summary] [--list-fields]
```

9.3.1 Obtaining and updating the script

The RDB web page `http://mu3edb0.psi.ch/rdb/` offers a download of `rdb.py`, which prefers the published copy on GitHub (`https://raw.githubusercontent.com/ursl/mu3eanca/master/db0/cdb2/rdb.py`) and falls back to the server checkout if GitHub is unreachable.

- `-h`, `--help` — print the help text and exit with status 0. With no arguments, or with an invalid option, the same text is printed and the status is 2.
- `--version` — print `rdb.py` and the script version (string `VERSION` in the source) and exit.
- `--update` — fetch the GitHub master file and replace the running script only if that copy has a higher `VERSION`. A remote file with no `VERSION`, or a local copy that is newer than GitHub, is left unchanged. No local JSON tree is required.

9.3.2 Downloading run records

The scripts allows for downloading run records from the CDB.

- `--sync` — download runrecords into the JSON tree (same endpoints as `syncJSON --rdb`). By default only significant runs are stored. First/last run and the run list (below) restrict which numbers are requested.
- `--host HOST` — REST host (default `mu3edb0` on port 5050, or `$MU3E_CDB_HOST`). A bare hostname uses port 5050 and path `/cdb`; `host:port` or a full `http(s)://` URL is accepted as given.
- `-a`, `--all` — with the REST download, store every run, not only significant ones.

9.3.3 JSON tree and sidecar cache

Files follow the block layout `runrecords/NNNN/runRecord_<run>.json`, where `NNNN` is the run number divided by 1000 and padded to four digits. Leftover flat files `runRecord_<run>.json` directly under `runrecords/` are still accepted.

Queries do not scan those JSON files. They read a flattened sidecar cache written next to them:

- `.rdb.index.jsonl` — one record per run;
- `.rdb.meta.json` — file count and modification-time fingerprint.

Each cache row holds all `BOR` and `EOR` keys that occur in the files, the *latest* `DataQuality` object in `Attributes` (historical `goodLinks` is stored as `dq_links`), and the latest `RunInfo` fields `Significant`, `Components`, `ComponentsOut`, `Class`, and `Comments`. A missing or incompatible cache is a hard error. A stale cache (JSON newer than the fingerprint, or a different file count) prints a warning on stderr and is still used.

- `-d`, `--dir DIR` — CDB root (a directory that contains `runrecords/`) or the `runrecords` directory itself. If omitted, the environment variable `MU3E_CDB` is used.
- `--rebuild` — rebuild the cache from the JSON tree. Rebuild is never automatic except after a REST download (below).
- `--list-fields` — print the flattened column names and exit.

9.3.4 Combined class

`BOR."Run Class"` is the shift-crew value and is not rewritten when classification is corrected later. `RunInfo.Class` is the bookkeeping override. Matching is case-insensitive. The raw values remain in the cache as `ri_class` and `bor_run_class`.

9.3.5 Filters

Unless noted, filters are combined with AND. Data-quality and `RunInfo` cuts apply to the latest `Attributes` entry of that type. Option abbreviation is disabled.

- `--significant` / `--not-significant` — latest `RunInfo.Significant` is `true` or `false`.
- `--class NAME` — combined class: `RunInfo.Class` if it is set, otherwise `BOR."Run Class"`. Repeatable (logical OR).
- `--dq EXPR` — latest data quality, e.g. `pixel=1` or `pixel!=-1`. Fields: `mu3e`, `beam`, `vertex`, `pixel`, `fibres`, `tiles`, `calibration`, `links`. Operators: `=`, `==`, `!=`, `>`, `>=`, `<`, `<=`. Repeatable.
- `--min-events` / `--max-events` — `EOR` event count.
- `-f` / `-l` — inclusive first/last run number (same letters as `syncJSON`). With a REST download these restrict which numbers are fetched.
- `-r` / `--r` / `--run LIST` — explicit run numbers and ranges, e.g. `226,4000-4010`. Not a prefix of rebuild. With a REST download these numbers are requested directly (no `findAll`).
- `--comment`, `--shift`, `--components`, `--components-out` — substring matches on EOR or RunInfo comments, BOR shift crew, and the corresponding `RunInfo` fields.

9.3.6 Output

Standard output is the list of matching run numbers.

- `--format FMT` — encoding of that list (default `csv`):
  - `csv` — comma-separated numbers on one line;
  - `json` — a JSON array of numbers;
  - `line` — one number per line;
  - `dump` — original JSON documents of the matching runs;
  - `count` — the number of matches only.
- `--summary` — write `rdb.py: N / T` to stderr after the result (N selected, T rows in the cache). Without this flag that line is omitted. Cache warnings and rebuild progress still go to stderr.

9.3.7 Examples.

```
python3 rdb.py --dir ~/data/mu3e/cdb --sync
python3 rdb.py --dir ~/data/mu3e/cdb --sync --all -f 4000 -l 4100
python3 rdb.py --dir ~/data/mu3e/cdb --rebuild
python3 rdb.py --dir ~/data/mu3e/cdb --significant
python3 rdb.py --dir ~/data/mu3e/cdb --class cosmic --significant --r 2000-3000
python3 rdb.py --dir ~/data/mu3e/cdb --class cosmic --significant --dq 'pixel!=-1'
python3 rdb.py --dir ~/data/mu3e/cdb -f 4000 -l 5000 --min-events 10000
python3 rdb.py --dir ~/data/mu3e/cdb --class beam --format json
python3 rdb.py --dir ~/data/mu3e/cdb --significant --format line --summary
python3 rdb.py --dir ~/data/mu3e/cdb -r 226 --format dump
python3 rdb.py --update
```

The directory argument may equally be `/data/mu3e/cdb/runrecords`. Quote `pixel!=-1` so the shell does not treat `!` as history expansion.

10 Detector Configuration Files

The `CDB` hosts a separate store for (possibly many and large) detector configuration files—opaque binary or text files such as pixel mask / TDAC sets. These are not to be confused with the structured conditions payloads managed through GTs/tags or the calibration file storage described in section 11. Access is via the HTTP routes mounted at `/detconfigs` on `mu3edb0` (see also section 3). Small CLI tools in `mu3eUtil/cdb/test` (`uploadDetConfig`, `downloadDetConfig`, `deleteDetConfig`) provide easy access and call the same routes. These tools are useful for testing/debugging and manual operations, but for production use, it is recommended to use the HTTP routes directly via (Python) scripts.

10.1 Design

Files are grouped by a string `tag` (for example `tdac_files_bu_06_10`). A tag may contain many files: each file is a separate MongoDB document with the same `tag` and its own `filename`. There is no GT/run-number/IOV lookup (different from the conditions database or the detcal database described in section 11). Downloading and deletion operate on the whole tag at once. Upload always inserts; there is no `replace` flag—re-uploading the same tag adds further documents unless the tag is deleted first.

As for `detcal`, uploads store file bytes in GridFS (bucket `detconfigBlobs`); the `detconfigs` document holds only metadata and a GridFS pointer. The upload middleware accepts files up to 100 MB.

10.2 Storage

Metadata live in the MongoDB collection `detconfigs`; file bytes for new uploads live in the GridFS bucket `detconfigBlobs` (parallel to `detcal`/`detcalBlobs`). Typical metadata fields:

- `tag` — grouping key for a configuration bundle;
- `filename` — original upload name;
- `size`, `uploadDate` — file size and timestamp;
- `blobStorage` (always `gridfs` for new uploads) and `blobGridFsId` — pointer to the GridFS object.

Legacy documents may still contain an inline `content` Binary / Buffer instead of a GridFS pointer. There are no MongoDB "subcollections"; a logical bundle is simply all documents that share a `tag`. Deleting a tag removes the metadata documents and any associated GridFS blobs.

10.3 Access

The Express router `db1/rest/routes/detconfigs.mjs` is mounted at `/detconfigs`. Typical base URL (reverse-proxied on `mu3edb0`): `http://mu3edb0/detconfigs`. Routes:

- `POST /detconfigs/upload` — multipart upload of a single file with form fields `file` and `tag`;
- `POST /detconfigs/uploadMany` — same for many files in one request (used by the web UI);
- `GET /detconfigs/downloadTag?tag=...` — ZIP archive of all files for that tag;
- `GET /detconfigs/detconfigTags` — distinct tags (plain text, one per line);
- `GET /detconfigs/findAll/detconfigsSummary` — JSON list of `{tag, count}`;
- `POST /detconfigs/uploadJSON` and `GET /detconfigs/downloadJSON/:tag` — JSON-oriented variants of the same collection;
- `DELETE /detconfigs/deleteDetconfigTag?tag=...` — remove all documents for a tag and their GridFS blobs (legacy alias `/detconfigs/deleteTag`).

The CLI tools use a subset: `uploadDetConfig` posts each matching file to `/upload`; `downloadDetConfig` uses `/downloadTag` (and `/detconfigTags` with `-listtags`); `deleteDetConfig` uses `/deleteDetconfigTag`.

10.4 Usage examples

The following examples upload mask files under a tag, list tags, download the ZIP bundle, and delete the tag. Paths and the tag name are illustrative (as used by `uploadDetConfig`).

10.4.1 Command line (`curl`)

Usage examples for the command line tool `curl`.

- Upload a single file under tag `tdac_files_bu_06_10`:

```
curl -X POST -F "tag=tdac_files_bu_06_10"
  -F "file=@/Users/ursl/Downloads/tdac_files_bu_06_10/mask_chip_0.bin"
  http://mu3edb0/detconfigs/upload
```

  Example response (plain text):

```
File uploaded successfully with ID: 6a72fcd4d7829f0204c9bd44
```

- Upload many files in one request (web-UI style):

```
curl -X POST -F "tag=tdac_files_bu_06_10"
  -F "file=@mask_chip_0.bin" -F "file=@mask_chip_1.bin"
  http://mu3edb0/detconfigs/uploadMany
```

- List all detconfigs tags (one per line):

```
curl -fsS "http://mu3edb0/detconfigs/detconfigTags"
```

- Summary with file counts (JSON):

```
curl -fsS "http://mu3edb0/detconfigs/findAll/detconfigsSummary"
```

Example response:

```
[{"tag":"tdac_files_bu_06_10","count":128}]
```

- Download all files for a tag as a ZIP archive:

```
curl -fSL --output tdac_files_bu_06_10.zip
  "http://mu3edb0/detconfigs/downloadTag?tag=tdac_files_bu_06_10"
```

- Delete all documents for a tag:

```
curl -fS -X DELETE
  "http://mu3edb0/detconfigs/deleteDetconfigTag?tag=tdac_files_bu_06_10"
```

Example response:

```
{"success":true,
  "message":"Successfully deleted 128 documents for tag: tdac_files_bu_06_10",
  "deletedCount":128}
```

Equivalent CLI wrappers (same HTTP surface):

```
uploadDetConfig --dir /Users/ursl/Downloads/tdac_files_bu_06_10
  -p mask_chip_ -t tdac_files_bu_06_10
downloadDetConfig --dir /tmp/out --tag tdac_files_bu_06_10
downloadDetConfig --listtags
deleteDetConfig --tag tdac_files_bu_06_10
```

10.4.2 Python (requests)

Example code using the `requests` library:

```python
import requests
from pathlib import Path

base = "http://mu3edb0/detconfigs"
basedir = Path("/Users/ursl/Downloads/tdac_files_bu_06_10")
tag = "tdac_files_bu_06_10"

# -- upload each matching file (like uploadDetConfig)
for path in sorted(basedir.glob("*mask_chip_*")):
    with open(path, "rb") as f:
        r = requests.post(
            f"{base}/upload",
            data={"tag": tag},
            files={"file": (path.name, f)},
        )
    print(path.name, r.status_code, r.text)

# -- list tags
r = requests.get(f"{base}/detconfigTags")
print(r.text)
```

```python
# -- summary
r = requests.get(f"{base}/findAll/detconfigsSummary")
print(r.json())

# -- download ZIP for the tag
r = requests.get(f"{base}/downloadTag", params={"tag": tag})
r.raise_for_status()
out_zip = Path(f"{tag}.zip")
out_zip.write_bytes(r.content)
print("wrote", out_zip, "bytes", len(r.content))

# -- delete the whole tag
r = requests.delete(f"{base}/deleteDetconfigTag", params={"tag": tag})
print(r.status_code, r.json())
```

11 Detector Calibration Files

The `CDB` hosts a separate store for (low-level) detector calibration files—arbitrary file content produced by calibration procedures (JSON, ROOT, binary, ...), not the structured conditions payloads managed through global tags and tags. Access is via the HTTP routes mounted at `/detcal` on `mu3edb0` (see also section 3). Storage in and/or access through the file-based `CDB` backend is not yet implemented.

11.1 Design

Each calibration series is identified by a string `name` (for example `pp0` for a pedestal-related set). Within a name, versions are keyed by a non-negative integer `run`: the run at which that file becomes valid. Retrieval is IOV-style by default, analogous to the conditiosn databaselogic: for a query run R, the server returns the document with the largest stored `run` $\leq R$. An exact match can be forced with the query parameter `exact=1`.

The pair `(name, run)` is unique. Re-uploading the same pair without specifying `replace=1` yields HTTP 409; with `replace=1` the previous GridFS blob is deleted and the metadata document is replaced. There is no GT/tag indirection and no payload schema validation—clients store and retrieve opaque files.

11.2 Storage

Metadata live in the MongoDB collection `detcal`, with a unique index on `(name, run)`. File bytes are always stored in the GridFS bucket `detcalBlobs` (parallel to conditions payloads / `payloadBlobs`). Each metadata document holds approximately:

- `name`, `run` — identity and IOV key;
- `filename` — original upload name;
- `comment`, `date` — optional annotation and ISO timestamp;
- `blobStorage` (always `gridfs`) and `blobGridFsId` — pointer to the GridFS object.

The upload middleware accepts files up to 100 MB.

11.3 Access

The Express router `db1/rest/routes/detcal.mjs` is mounted at `/detcal`. Typical base URL (reverse-proxied on `mu3edb0`): `http://mu3edb0/detcal`. Routes:

- `POST /detcal/upload` — multipart form fields `file`, `name`, `run`, optional `comment`; optional `replace=1` (form or query);
- `GET /detcal/:name` — JSON list of stored runs for that name (newest first; metadata only);

- `GET /detcal/:name/:run/meta` — metadata for the IOV-selected (or exact) document;
- `GET /detcal/:name/:run` — download file bytes (attachment; headers `X-Detcal-Name`, `X-Detcal-Run`);
- `DELETE /detcal/:name/:run` — delete the exact `(name, run)` and its GridFS blob.

11.4 Usage examples

The following examples upload two files under the name `pp0` at runs 5700 and 7700, list the stored entries, and download the file valid at run 6000 (which resolves to the run-5700 document).

11.4.1 Command line (`curl`)

Usage examples for the command line tool `curl`.

- Upload file `detector0.json` for run 5700:

```
curl -X POST -F "name=pp0" -F "run=5700"
  -F "file=@/Users/ursl/mu3e/software/pixel_scripts/config/detector0.json"
  http://mu3edb0/detcal/upload
```

  Example response:

```
{"message":"Detcal uploaded","detcal":{"name":"pp0","run":5700,
  "filename":"detector0.json","comment":"","date":"2026-08-05T09:05:25.323Z",
  "blobStorage":"gridfs","blobGridFsId":"6a72fcd4d7829f0204c9bd44"}}
```

- Upload file `detector1.json` for run 7700:

```
curl -X POST -F "name=pp0" -F "run=7700"
  -F "file=@/Users/ursl/mu3e/software/pixel_scripts/config/detector1.json"
  http://mu3edb0/detcal/upload
```

  Example response:

```
{"message":"Detcal uploaded","detcal":{"name":"pp0","run":7700,
  "filename":"detector1.json","comment":"","date":"2026-08-05T09:05:42.214Z",
  "blobStorage":"gridfs","blobGridFsId":"6a72fce6d7829f0204c9bd47"}}
```

- List all stored runs for `pp0` (JSON to a local file):

```
curl "http://mu3edb0/detcal/pp0" > pp0
```

  Example contents in file `pp0`:

```
moor>cat pp0
["name":"pp0","run":7700,"filename":"detector1.json","comment":"","date":"2026-08-0
 "name":"pp0","run":5700,"filename":"detector0.json","comment":"","date":"2026-08-0
```

- Download the file valid at run 6000 (`-OJ` writes the attachment filename from the response headers; here `detector0.json`):

```
curl -OJ "http://mu3edb0/detcal/pp0/6000"
```

- Overwrite an existing `(name, run)` (without `replace=1` the same upload returns HTTP 409):

```
curl -X POST -F "name=pp0" -F "run=5700" -F "replace=1"
  -F "file=@/Users/ursl/mu3e/software/pixel_scripts/config/detector0.json"
  http://mu3edb0/detcal/upload
```

  Example response:

```
{"message":"Detcal replaced","detcal":{"name":"pp0","run":5700,
  "filename":"detector0.json","comment":"","date":"...",
  "blobStorage":"gridfs","blobGridFsId":"..."}}
```

- Exact run match on download: only succeed if a document with `run=6000` exists (with the uploads above this returns 404; `run=5700` succeeds and returns `detector0.json`):

```
curl -OJ "http://mu3edb0/detcal/pp0/6000?exact=1"   # 404
curl -OJ "http://mu3edb0/detcal/pp0/5700?exact=1"   # detector0.json
```

- Exact run match on metadata (same selection rule):

```
curl "http://mu3edb0/detcal/pp0/6000/meta?exact=1"  # 404
curl "http://mu3edb0/detcal/pp0/5700/meta?exact=1"
```

  Example response for the successful metadata call:

```
{"name":"pp0","run":5700,"filename":"detector0.json","comment":"",
  "date":"2026-08-05T09:05:25.323Z","blobStorage":"gridfs",
  "blobGridFsId":"6a72fcd4d7829f0204c9bd44"}
```

11.4.2 Python (`requests`)

Example code using the `requests` library:

```python
import requests

base = "http://mu3edb0/detcal"
basedir = "/Users/ursl/mu3e/software/pixel_scripts/config"

# -- upload (run 5700)
with open(f"{basedir}/detector0.json", "rb") as f:
    r = requests.post(
        f"{base}/upload",
        data={"name": "pp0", "run": "5700"},
        files={"file": ("detector0.json", f)},
    )
print(r.status_code, r.json())

# -- upload (run 7700)
with open(f"{basedir}/detector1.json", "rb") as f:
    r = requests.post(
        f"{base}/upload",
```

```
        data={"name": "pp0", "run": "7700"},
        files={"file": ("detector1.json", f)},
    )
print(r.status_code, r.json())

# -- list stored runs for name pp0
r = requests.get(f"{base}/pp0")
open("pp0", "w").write(r.text)
print(r.json())

# -- download IOV-selected file for run 6000 (-> run 5700)
r = requests.get(f"{base}/pp0/6000")
r.raise_for_status()
# Content-Disposition: attachment; filename="detector0.json"
fname = r.headers.get("Content-Disposition",
    "").split("filename=")[-1].strip('"')
if not fname:
    fname = "detcal_pp0_6000.bin"
open(fname, "wb").write(r.content)
print("wrote", fname, "resolved run", r.headers.get("X-Detcal-Run"))

# -- replace an existing (name, run)
with open(f"{basedir}/detector0.json", "rb") as f:
    r = requests.post(
        f"{base}/upload",
        data={"name": "pp0", "run": "5700", "replace": "1"},
        files={"file": ("detector0.json", f)},
    )
print(r.status_code, r.json()) # 200, message "Detcal replaced"

# -- exact run match: download / metadata
r = requests.get(f"{base}/pp0/6000", params={"exact": "1"})
print(r.status_code) # 404 (no document with run==6000)
r = requests.get(f"{base}/pp0/5700", params={"exact": "1"})
r.raise_for_status()
open("detector0.json", "wb").write(r.content)

r = requests.get(f"{base}/pp0/5700/meta", params={"exact": "1"})
print(r.json())
```

References

[1] Urs Langenegger, A conditions database for the mu3e experiment, `https://github.com/ursl/mu3eanca/blob/master/db0/cdb2/slides/Mu3e-Note-0099-ConditionsDatabase.pdf`, 2023.

[2] Urs Langenegger, `nginx-proxy-common.conf` and `nginx.conf`, `https://github.com/ursl/mu3eanca/tree/master/db2`, 2026.

[3] Urs Langenegger, mu3eanca/db1/rest, `https://github.com/ursl/mu3eanca/tree/master/db1/rest`.